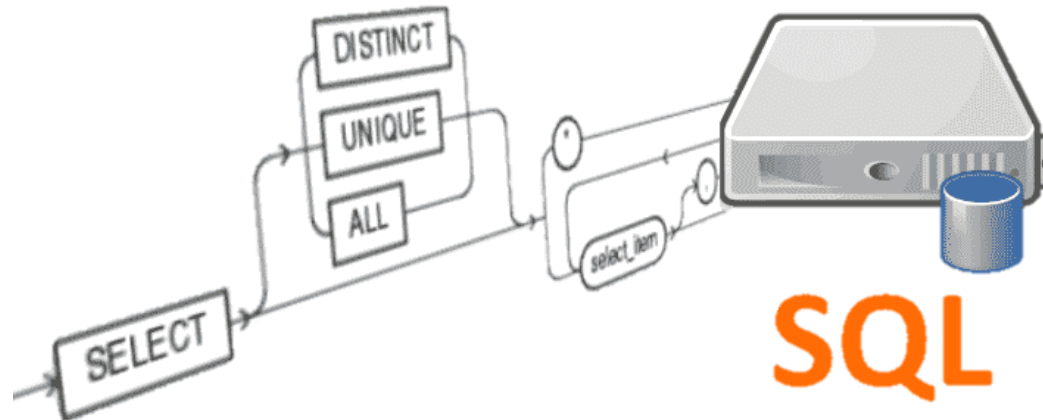


SQL

Structured Query Language

(Lenguaje de Consulta Estructurada)

1



Lenguaje de Definición de Datos

DDL (Data Definition Language): conjunto de comandos para definir, modificar y eliminar la estructura de los objetos de una base de datos.

CREATE: Este comando se utiliza para crear objetos en la base de datos, como tablas, vistas, índices, esquemas y procedimientos almacenados.

CREATE DATABASE nombre_base_datos;

CREATE DATABASE librería

Este comando crea una base de datos llamada librería.

Lenguaje de Definición de Datos

Crear una Tabla

```
CREATE TABLE nombre_tabla  
( nombre_columna1 tipo_dato restriccion,  
  nombre_columna2 tipo_dato restriccion,  ...);
```

```
CREATE TABLE empleado ( id_empleado INT PRIMARY KEY,  
                           nombre VARCHAR(50),  
                           sector VARCHAR(50),  
                           sueldo DECIMAL(10, 2))
```

Crea una tabla llamada empleado con columnas id_empleado, nombre, sector y sueldo.

Lenguaje de Definición de Datos

```
CREATE TABLE clientes (id_cliente INT PRIMARY KEY,  
                           nombre VARCHAR(50) NOT NULL,  
                           email VARCHAR(100) UNIQUE,  
                           fecha_registro DATE DEFAULT CURRENT_DATE )
```

```
CREATE TABLE pedido(  
  id_pedido INT PRIMARY KEY,  
  id_cliente INT,  
  fecha_pedido DATE,  
  monto_total DECIMAL(10, 2),  
  CONSTRAINT fk_cliente FOREIGN KEY  
  (id_cliente) REFERENCES clientes(id_cliente));
```

Lenguaje de Definición de Datos

ALTER modifica la estructura de un objeto de la base de Datos. Puede agregar, modificar o eliminar columnas en una tabla.

ALTER TABLE Empleados **ADD** fechaNacimiento **DATE**

Este comando añade una columna fechaNacimiento a la tabla empleados

DROP elimina objetos de la base de datos. La eliminación es permanente y no se puede recuperar sin un respaldo

DROP TABLE empleados

Este comando elimina la tabla empleados de la base de datos.

Tipos de datos en SQL

- **INTEGER**: entero binario con signo de palabra completa (31 bits de precisión).
- **SMALLINT**: entero binario con signo de media palabra (15 bits de precisión).
- **DECIMAL (p[,q])**: número decimal con signo de p dígitos de precisión, q decimales. Si q se omite, se asume que vale 0.
- **FLOAT**: numérico con signo de doble palabra y coma flotante.
- **CHAR(n)**: cadena de caracteres de longitud fija, de longitud n.
- **VARCHAR(n)**: cadena de caracteres de longitud variable, de longitud máxima n.

Las restricciones (constraints)

Son reglas que se aplican a las columnas de una tabla para asegurar la integridad de los datos. Las restricciones pueden aplicarse al crear la tabla o al modificar una tabla existente.

PRIMARY KEY (PK): Define una columna o un conjunto de columnas que identifican de forma única cada fila de la tabla. Una tabla solo puede tener una PRIMARY KEY, y los valores en esta columna no pueden ser nulos ni repetidos

```
CREATE TABLE empleado (id_empleado INT PRIMARY KEY,  
                           nombre VARCHAR(50));
```

Las restricciones (constraints)

FOREIGN KEY (FK): Crea una relación entre dos tablas, conectando la FK de una tabla y la PK de otra. La **integridad referencial** garantiza que el valor FK de una tabla exista en la tabla de referencia como PK

```
CREATE TABLE sector ( id_sector INT PRIMARY KEY,  
                        sector VARCHAR(50));
```

```
CREATE TABLE empleado (  
    id_empleado INT PRIMARY KEY,  
    nombre VARCHAR(50),  
    id_sector INT,  
    FOREIGN KEY (id_sector) REFERENCES sector (id_sector));
```


Las restricciones (constraints)

Se debe tener en cuenta que para crear una FK en una tabla, debe existir el atributo clave (PK) en la tabla a la que se quiere hacer referencia, de lo contrario el motor da un error en la creación de la tabla.

```
CREATE TABLE empleado (  
  id_empleado INT PRIMARY KEY,  
  nombre VARCHAR(50),  
  id_sector INT,  
  FOREIGN KEY (id_sector) REFERENCES sector (id_sector));
```

```
CREATE TABLE sector ( id_sector INT PRIMARY KEY,  
                        sector VARCHAR(50));
```

#	Time	Action	Message
✓ 1	18:20:14	Apply changes to prueba	Changes applied
✗ 2	18:20:28	CREATE TABLE empleado (id_empleado INT PRIMARY KEY, nombre VA...	Error Code: 1824. Failed to open the referenced table 'sector'

Las restricciones (constraints)

UNIQUE: Asegura que todos los valores de una columna sean únicos. A diferencia de PRIMARY KEY, estas columnas pueden tener valores nulos

NOT NULL: Garantiza que una columna no puede tener valores nulos. Asegura que los campos importantes siempre tengan un valor

```
CREATE TABLE cliente (id_cliente INT PRIMARY KEY,  
                        nombre VARCHAR(50) NOT NULL,  
                        cuil VARCHAR(15) UNIQUE);
```


Ejemplos - Restricciones

```
CREATE TABLE empleados (  
  id_empleado INT PRIMARY KEY, -- asegura unicidad  
  nombre VARCHAR(50) NOT NULL, -- asegura que el nombre no sea nulo  
  email VARCHAR(100) UNIQUE, -- asegura que el email no se repita  
  sueldo DECIMAL(9, 2) CHECK (sueldo > 0), -- asegura que el sueldo sea positivo  
  id_depto INT,  
  FOREIGN KEY (id_depto) REFERENCES  
  departamentos (id_depto)); -- conecta con otra tabla
```

Ejemplos - Restricciones

```
CREATE TABLE producto ( Id_producto INT,  
                           Nombre VARCHAR(30) NOT NULL,  
                           Descripcion VARCHAR(255),  
                           Precio DECIMAL(9,2),  
                           NroProv INT,  
CONSTRAINT PK_ producto PRIMARY KEY (Id_producto),  
CONSTRAINT FK_Proveedor FOREIGN KEY (NroProv)  
REFERENCES Proveedor(Id_proveedor) );
```

El uso de **CONSTRAINT** es opcional en SQL.

Resumen - Restricciones

PRIMARY KEY: Unicidad y no nulo para identificar filas.

FOREIGN KEY: Enlace entre tablas para asegurar integridad referencial.

UNIQUE: Exclusividad en valores no nulos en una columna.

NOT NULL: Evita valores nulos en una columna.

CHECK: Define condiciones específicas para los valores.

DEFAULT: Asigna valores predeterminados en ausencia de datos.

Restricciones que se aplican con el Comando CREATE:

- Clave Primaria (**PRIMARY KEY**)
- Unicidad (**UNIQUE**)
- Obligatoriedad (**NOT NULL**)
- Integridad Referencial (**FOREIGN KEY**)
- Restricciones de Rechazo: Verificación (**CHECK**)

```
CREATE TABLE alumno ( idAlumno    INT NOT NULL,  
                        nombre      VARCHAR(45) NOT NULL,  
                        NroSocio    INT,  
                        Ingreso     DATE,  
                        PRIMARY KEY (idAlumno),  
                        UNIQUE INDEX (NroSocio));
```



DDL - ALTER

ALTER modifica la estructura de una tabla. Puede agregar, modificar, o eliminar columnas, y también cambiar las restricciones de la tabla.

Agrega la columna fecha_contratacion de tipo DATE a la tabla empleado

ALTER TABLE empleado **ADD** fecha_contratacion **DATE**

Modifica una columna

Cambia el tipo de dato de la columna sueldo a DECIMAL(15, 2) y agrega la restricción NOT NULL.

ALTER TABLE empleado **MODIFY** sueldo **DECIMAL(15, 2) NOT NULL**



DDL - ALTER

Renombrar una columna

ALTER TABLE nombre_tabla

CHANGE COLUMN nom_colum_anterior **TO** nom_colum_nuevo

ALTER TABLE empleado **CHANGE COLUMN** nombre **TO** nomb_completo

cambia el nombre de la columna nombre a nomb_completo

Eliminar una columna

ALTER TABLE empleado **DROP COLUMN** fecha_contratación

Elimina la columna fecha_contratacion de la tabla empleado.



DDL - ALTER

Agregar una restricción

ALTER TABLE empleado **ADD CONSTRAINT** fk_sector **FOREIGN KEY** (id_sector) **REFERENCES** sector (id_sector)

agrega una restricción de clave foránea llamada fk_sector que conecta id_sector de empleado con id_sector de sector

Eliminar una restricción

ALTER TABLE empleado **DROP CONSTRAINT** fk_sector

Elimina la restricción fk_sector de la tabla empleado.

Las restricciones (constraints)

Si se desea agregar restricciones de unicidad o de obligatoriedad en una tabla existente, se debe tener en cuenta que, si la columna elegida no cumple con las restricciones va a dar error.

```
CREATE TABLE empleado ( id_empleado int primary key,  
nombre varchar(50) not null, domicilio varchar(50), id_sector int)
```

```
insert into empleado values (1,'empleado1',1, default),  
(2,'empleado2',1, default), (3,'empleado3',1, default), ,...
```

```
Alter table empleado add constraint u_id_sector unique (id_sector);
```

```
Alter table empleado modify domicilio varchar(50) not null;
```

✖ 17 18:46:25 ALTER TABLE empleado ADD CONSTRAINT unique_id_sector UNIQUE... Error Code: 1062. Duplicate entry '1' for key 'empleado.unique_id_sector'

✖ 18 18:51:42 ALTER TABLE empleado MODIFY domicilio VARCHAR(50) NOT NULL Error Code: 1138. Invalid use of NULL value



DDL - ALTER

Renombrar una tabla

ALTER TABLE empleado **RENAME TO** personal

Cliente(id_cliente, cliente, direccion)

Agregar una columna email, agregar una restricción UNIQUE a email, cambiar la el nombre de columna cliente por nombre, eliminar la columna dirección, renombrar la tabla de cliente a usuario

ALTER TABLE cliente **ADD** email **VARCHAR**(100);

ALTER TABLE cliente **ADD CONSTRAINT** unique_email **UNIQUE** (email);

ALTER TABLE cliente **CHANGE COLUMN** cliente **TO** nombre;

ALTER TABLE cliente **DROP COLUMN** direccion;

ALTER TABLE cliente **RENAME TO** usuario;

Manipulación de datos - INSERT

INSERT agrega nuevos registros (filas) en una tabla de una base de datos.

Insertar una fila especificando todos los valores para cada columna. Los valores deben estar en el mismo orden de las columnas de la tabla

INSERT INTO nombre_tabla **VALUES** (valor1, valor2, valor3, ...)

Supongamos que tenemos una tabla llamada clientes con las columnas id_cliente, nombre, y edad

INSERT INTO clientes **VALUES** (1, 'Juan Pérez', 30);

Manipulación de datos - INSERT

Insertar una fila especificando columnas. Si alguna columna acepta nulos o tiene configurado un valor por defecto, se puede especificar solo las columnas en las que van a insertar datos.

```
INSERT INTO nombre_tabla (columna1, columna2, ...)  
VALUES (valor1, valor2, ...)
```

Continuando con la tabla clientes, solo se agrega nombre y edad, idCliente se genera automáticamente

```
INSERT INTO clientes (nombre, edad) VALUES ('Ana Gómez', 25);
```

Manipulación de datos - INSERT

Insertar varias filas en una sola consulta.

```
INSERT INTO nombre_tabla (columna1, columna2, ...)  
VALUES (valor1a, valor2a, ...),  
        (valor1b, valor2b, ...),  
        ...,  
        (valor1n, valor2n, ...);
```

Agregar tres clientes en una sola consulta

```
INSERT INTO clientes (nombre, edad)  
VALUES ('Carlos López', 40),  
        ('Maria Sanchez', 35),  
        ('Jose Martinez', 28);
```

Manipulación de datos - INSERT

Insertar resultados de otra consulta (INSERT INTO ... SELECT).

```
INSERT INTO tabla_destino (columna1, columna2, ...)  
SELECT columna1, columna2, ...  
FROM tabla_origen WHERE condición
```

Insertar los clientes menores de 30 años de cliente a **cliente_joven**

```
INSERT INTO cliente_joven (id_cliente, nombre, edad)  
SELECT id_cliente, nombre, edad FROM cliente WHERE edad < 30;
```

En caso de que las dos tablas tengan la misma estructura se puede usar

```
INSERT INTO cliente_joven  
SELECT * FROM cliente WHERE edad < 30;
```


Manipulación de datos - INSERT

Insertar datos utilizando valores por defecto (DEFAULT). Si la tabla tiene columnas configuradas con valores por defecto se puede usar DEFAULT

```
INSERT INTO nombre_tabla (columna1, columna2, columna3)  
VALUES (valor1, DEFAULT, valor3)
```

Agregar un cliente, con la columna ciudad configurada por defecto con un valor predeterminado

```
INSERT INTO cliente (id_cliente, nombre, ciudad )  
VALUES (3, 'Luis Romero', DEFAULT);
```

Manipulación de datos - INSERT

Resumen

INSERT INTO ... VALUES: Añadir una sola fila.

INSERT INTO ... (col) VALUES: Añadir fila(s) especificando columnas.

INSERT INTO ... VALUES (), (), ...: Añadir múltiples filas.

INSERT INTO ... SELECT: Añadir datos basados en resultados de consulta.

INSERT INTO ... VALUES (... , DEFAULT, ...): Insertar valores con por default

Manipulación de datos - UPDATE

UPDATE modifica los valores de una o más columnas de una tabla.

Actualizar una fila específica usando WHERE. Con WHERE se define cuál fila debe actualizarse, pueden ser mas de una columna al mismo tiempo

UPDATE nombre_tabla **SET** columna1 = valor1, columna2 = valor2,
WHERE condición

Actualiza la ciudad y el teléfono del cliente 3 (id_cliente = 3)

UPDATE cliente **SET** ciudad = 'Capital', teléfono=22334455
WHERE id_cliente = 3;

Manipulación de datos - UPDATE

Actualizar varias filas con una condición. De manera similar a la anterior, las filas a modificar dependen de la condición en la cláusula WHERE

Aumentar el sueldo en 30 % a los empleados del sector ventas

UPDATE empleado **SET** sueldo = sueldo * 1.3
WHERE sector = 'ventas'

Actualizar todas las filas de una tabla (sin WHERE). Esta manera tan practica como peligrosa, se debe usar con mucho cuidado

Aumentar el sueldo en 30 % de todos los empleados

UPDATE empleado **SET** sueldo = sueldo * 1.3

Manipulación de datos - UPDATE

Actualizar datos usando JOIN: A veces es útil actualizar datos de una tabla basándote en información de otra tabla, lo cual puedes hacer con JOIN en el comando UPDATE

Actualizar el estado de clientes a 'activo' solo para aquellos que tienen una suscripción activa.

```
UPDATE clientes SET estado = 'activo'  
FROM clientes c  
INNER JOIN suscripciones s ON c.id_cliente = s.id_cliente  
WHERE s.estado = 'activa';
```

Manipulación de datos - UPDATE

Resumen

UPDATE ... SET ... WHERE: Actualizar una o más columnas en filas específicas.

UPDATE ... SET ...: Actualizar todas las filas de una tabla (sin WHERE).

UPDATE ... SET ... WHERE ... (subconsulta): Actualizar datos basados en resultados de una subconsulta.

UPDATE ... JOIN: Actualizar columnas en función de condiciones y valores de otras tablas.

Ejemplo - UPDATE con INNER JOIN

Actualizar el estado de los clientes a "con pedido pendiente" si tienen al menos un pedido con estado "pendiente".

Clientes(id_cliente, nombre, estado)

Pedidos(id_pedido, id_cliente, estado_pedido)

UPDATE clientes **SET** estado = 'con pedido pendiente'

FROM clientes c **INNER JOIN** pedidos p **ON** c.id_cliente = p.id_cliente

WHERE p.estado_pedido = 'pendiente'

La condición **WHERE** p.estado_pedido = 'pendiente' asegura que solo se actualizarán los clientes que tienen pedidos pendientes, cambiando su estado

Ejemplo - UPDATE con INNER JOIN

Aumentar el límite de crédito en 20% a los clientes que tienen un gasto total mayor a \$5000

Clientes(id_cliente, nombre, limite)

Transacciones(id_transaccion, id_cliente, monto)

UPDATE clientes **SET** limite = limite * 1.2 **FROM** clientes c **INNER JOIN**

(SELECT id_cliente c, **SUM**(monto) **FROM** transacciones t

GROUP BY id_cliente **HAVING SUM**(monto) > 5000) **AS** gastosT

ON c.id_cliente = gastosT.id_cliente

La subconsulta calcula el gasto de cada cliente y filtra los mayor a \$5000

Ejemplo - UPDATE con INNER JOIN

Cambiar el estado de los productos a "Prioritario" si el stock es menor que la cantidad

productos(id_producto, nombre, stock, estado)

pedido(id_producto, cantidad, fecha)

UPDATE productos **SET** estado = 'Prioritario'

FROM productos p

INNER JOIN pedido pe **ON** p.id_producto = pe.id_producto

WHERE p.stock < pe.cantidad

Manipulación de datos - DELETE

DELETE elimina registros específicos de una tabla.

Eliminar filas específicas con una condición (WHERE). La cláusula WHERE especifica qué registros serán eliminados. Si omite (WHERE) se eliminarán todas las filas de la tabla

DELETE FROM nombre_tabla **WHERE** condición

Eliminar el cliente 3 (id_cliente = 3)

DELETE FROM clientes **WHERE** id_cliente = 3

Manipulación de datos - DELETE

Eliminar varias filas que cumplen con una condición específica. Se usa los operadores lógicos (**AND, OR**) o de comparación **<, >, IN**, etc.

Eliminar los empleados mayores de 70 años y que cobren mas de 990000

```
DELETE FROM empleado  
WHERE edad > 70 AND sueldo > 990000
```

Eliminar todas las filas de una tabla (sin WHERE). Este comando elimina todas las filas y no se pueden recuperar sin un respaldo

```
DELETE FROM cliente
```

Eliminar todos los registros de la tabla clientes

Manipulación de datos - DELETE

Eliminar datos usando JOIN. Eliminar datos de una tabla basándose en criterios que involucran múltiples tablas.

Eliminar los clientes que no tienen una suscripción activa

```
DELETE cliente FROM clientes c  
LEFT JOIN suscripciones s ON c.id_cliente = s.id_cliente  
WHERE s.id_cliente IS NULL
```

Este comando elimina a todos los clientes que no tienen registros correspondientes en suscripciones (no tienen una suscripción activa).

Manipulación de datos - DELETE

Resumen

DELETE FROM ... WHERE: Eliminar filas específicas que cumplen una condición.

DELETE FROM ... WHERE condición_lógica: Eliminar varias filas que cumplen condiciones múltiples.

DELETE FROM ...: Eliminar todas las filas de una tabla (sin WHERE).

DELETE FROM ... JOIN: Eliminar filas usando condiciones basadas en otras tablas

Ejemplo - DELETE con INNER JOIN

Eliminar empleados que no están en ningún proyecto.

Empleado(id_empleado, nombre, departamento)
Asignado (id_proyecto, id_empleado)

```
DELETE e  
FROM empleado e  
LEFT JOIN Asignado a ON e.id_empleado = a.id_empleado  
WHERE a.id_empleado IS NULL
```

Aquí usamos un LEFT JOIN para identificar a los empleados que no tienen coincidencias en la tabla Asignados (su valor es NULL), lo cual significa que no están asignados a ningún proyecto.

Ejemplo - DELETE con INNER JOIN

Eliminar pedidos con productos descontinuados. (disponible = FALSE)

Pedido(id_pedido, id_producto, cantidad)

Producto(id_producto, nombre, disponible)

DELETE p

FROM pedido p

INNER JOIN producto pr **ON** p.id_producto = pr.id_producto

WHERE pr.disponible = **FALSE**

En este caso, usamos un INNER JOIN para encontrar todos los registros de pedidos que tienen productos cuyo campo disponible está en FALSE.

Ejemplo - DELETE con INNER JOIN

Eliminar estudiantes sin calificaciones en ninguna materia

Estudiante (id_estudiante, nombre)

Calificacion (id_estudiante, id_materia, calificación)

DELETE e

FROM estudiante e

LEFT JOIN calificacion c **ON** e.id_estudiante = c.id_estudiante

WHERE c.id_estudiante **IS NULL**

LEFT JOIN identifica los estudiantes que no tienen calificaciones.

Manipulación de datos - Truncate

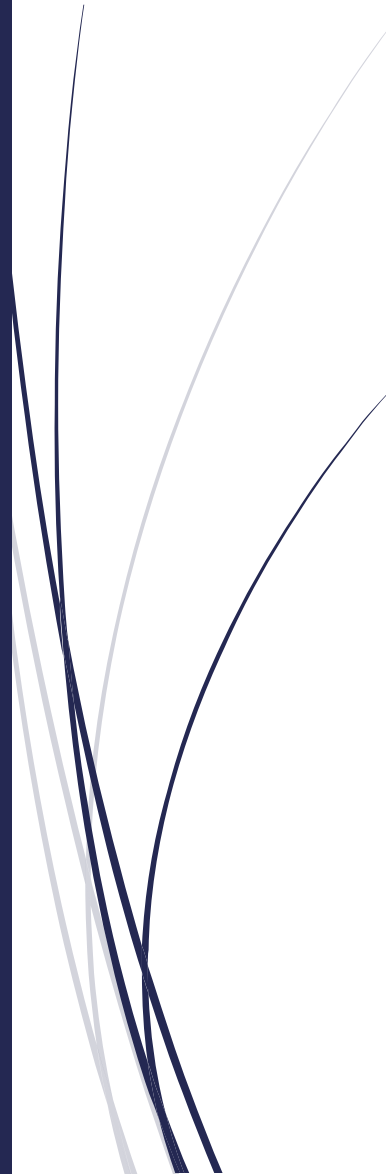
TRUNCATE elimina todos los registros de la tabla sin afectar su estructura (no afecta a las columnas)

DELETE, elimina las filas una por una, mientras que **TRUNCATE** elimina los datos en bloques, liberando el espacio de almacenamiento ocupado por la tabla y reinicia cualquier columna de identidad (autoincremental) a su valor inicial.

TRUNCATE TABLE nombre_tabla

Eliminar los datos de tabla cliente

TRUNCATE TABLE cliente



Muchas Gracias